

Relatório Final — Projeto Interdisciplinar 6º Semestre

EntreMentes: Plataforma de Monitoramento de Saúde Mental e Bem-Estar Acadêmico

Instituição: Faculdade de Tecnologia de Franca — FATEC Franca

Curso: Tecnologia em Desenvolvimento de Software Multiplataforma — DSM

Período: 1º Semestre de 2026 (março – junho)

Grupo: Gabriel Fillip | Leonardo Cássio

Professor Responsável (Disciplina-Chave): Prof. Me. Alexandre Gomes

Repositório GitHub: <https://github.com/Leonardo-Cassio/EntreMentes>

Vídeo Demonstração (YouTube): <https://youtu.be/p0qIAkJlquc>

Sumário

1. [Resumo](#)
 2. [Introdução](#)
 3. [Definição do Escopo e Requisitos](#)
 4. [Modelagem e Arquitetura do Sistema](#)
 5. [Desenvolvimento Multiplataforma](#)
 6. [Computação em Nuvem II](#)
 7. [Mineração de Dados](#)
 8. [Histórico de Sprints](#)
 9. [Resultados e Considerações Finais](#)
 10. [Referências](#)
-

1. Resumo

O **EntreMentes** é uma plataforma digital de monitoramento de saúde mental e bem-estar acadêmico voltada a estudantes universitários. Desenvolvida ao longo do 6º semestre do curso de DSM na FATEC Franca, a plataforma permite que estudantes registrem diariamente seu estado emocional e hábitos de vida (sono, tempo de tela, exercício físico, nível de estresse), e aplica algoritmos de mineração de dados para identificar automaticamente o perfil comportamental de cada usuário, fornecendo insights personalizados.

O sistema é composto por quatro camadas integradas: uma API REST em Node.js com Express e Prisma (backend), uma interface web em React com Vite (frontend web), um aplicativo mobile em React Native com Expo SDK 54 (frontend mobile) e um serviço independente em Python/Flask para classificação por K-Means. A comunicação entre o backend e o serviço de mineração é assíncrona via fila de mensagens **BullMQ + Redis**, garantindo que o usuário nunca aguarde a classificação ao salvar um registro. Toda a infraestrutura — API, mining-service, PostgreSQL e Redis — está implantada na plataforma em nuvem Railway com HTTPS, restart automático e deploy contínuo via GitHub.

Tecnologias principais: Node.js, Express, Prisma, PostgreSQL, React, React Native, Expo, Python, Flask, scikit-learn, BullMQ, Redis, Railway.

2. Introdução

2.1 Contexto e Motivação

A saúde mental de estudantes universitários é um tema de crescente relevância no cenário acadêmico brasileiro. Segundo levantamentos recentes, uma parcela expressiva de universitários relata sintomas de ansiedade, depressão e esgotamento ao longo da graduação — quadros frequentemente agravados pela pressão de provas, prazos e pela rotina acadêmica intensa.

O EntreMentes surge como resposta a essa demanda: uma ferramenta que coloca o estudante no centro do seu próprio processo de autoconhecimento emocional, permitindo o registro consistente de dados sobre seu bem-estar e devolvendo ao usuário uma análise inteligente sobre seus padrões comportamentais.

2.2 Problema

Estudantes universitários, em geral, não acompanham de forma estruturada seus padrões emocionais ao longo do semestre. A ausência de ferramentas acessíveis, não-clínicas e integradas à rotina acadêmica dificulta a percepção precoce de sinais de alerta, como deterioração do sono, aumento do estresse ou redução da atividade física — fatores comprovadamente relacionados ao rendimento acadêmico e à saúde mental.

2.3 Solução Proposta

Uma plataforma multiplataforma (web + mobile) que permite:

1. Registro diário simplificado de humor e hábitos de vida;
2. Visualização do histórico emocional com gráficos e métricas;
3. Classificação automática do estudante em um perfil comportamental via K-Means;
4. Exibição de insights e recomendações personalizadas baseadas no perfil identificado.

Disclaimer: o EntreMentes **não substitui** acompanhamento profissional de saúde mental. Toda análise gerada pela plataforma tem caráter informativo.

3. Definição do Escopo e Requisitos

3.1 Escopo

O sistema FAZ:

- Cadastro e autenticação de usuários via JWT
- Registro diário de humor (escala 1–5 com emojis), sono, tempo de tela, exercício físico, estresse e desempenho acadêmico
- Histórico emocional com visualização cronológica
- Dashboard com gráficos de evolução de humor e métricas agregadas
- Classificação comportamental automática via K-Means (K=4)
- Exibição de perfil comportamental com insights e recomendações
- Página de estatísticas com análises agregadas do usuário
- Gerenciamento de conta (edição de perfil, troca de senha, exclusão de conta)
- API REST documentada (Swagger/OpenAPI)
- Deploy completo em nuvem (Railway): API, mining-service, PostgreSQL e Redis
- Mensageria assíncrona com fila de jobs (BullMQ + Redis) para classificação em background
- Painel visual de monitoramento da fila (Bull Board em </admin/queues>)

O sistema NÃO FAZ:

- Diagnóstico psicológico profissional
- Chat ou sessão de terapia online
- Integração com sistemas de saúde (prontuário, SUS, etc.)
- Notificações por SMS

- Sistema de pagamento
- Funcionalidade offline no mobile

3.2 Requisitos Funcionais

ID	Requisito	Prioridade	Status
RF01	Cadastro de usuário (nome, e-mail, senha)	Alta	 Implementado
RF02	Autenticação via e-mail e senha com JWT	Alta	 Implementado
RF03	Registro de humor diário (escala 1–5 com emojis)	Alta	 Implementado
RF04	Nota textual opcional no registro	Média	 Implementado
RF05	Questionário de bem-estar (sono, estresse, exercício, tela, desempenho)	Alta	 Implementado
RF06	Armazenamento de registros vinculados ao usuário e data	Alta	 Implementado
RF07	Histórico em ordem cronológica	Alta	 Implementado
RF08	Gráficos de evolução de humor ao longo do tempo	Alta	 Implementado
RF09	Dashboard com dados agregados (média, distribuição, correlações)	Alta	 Implementado
RF10	Clusterização K-Means nos dados para identificar perfis	Alta	 Implementado
RF11	Classificação do usuário em perfil comportamental	Alta	 Implementado
RF12	Visualização do perfil comportamental identificado	Média	 Implementado
RF13	Edição de dados do perfil do usuário	Baixa	 Implementado
RF14	API REST documentada para comunicação frontend–backend	Alta	 Implementado

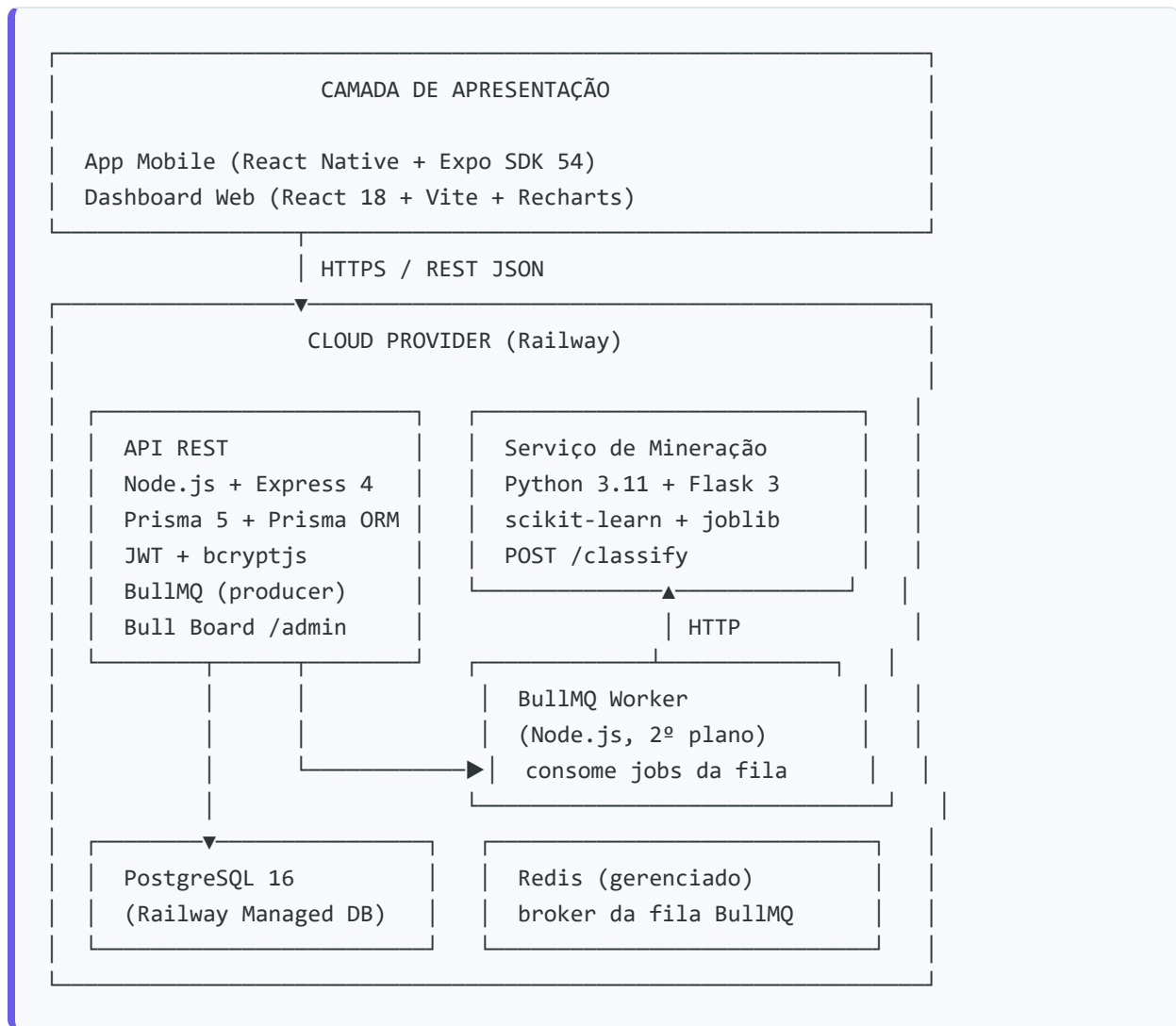
3.3 Requisitos Não Funcionais

ID	Requisito	Categoria	Status
RNF01	Sistema disponível online 24/7 via deploy em nuvem	Disponibilidade	✓ Railway
RNF02	API responde em no máximo 2 segundos	Performance	✓ Testado
RNF03	Senhas armazenadas com hash bcrypt	Segurança	✓ bcryptjs
RNF04	Comunicação cliente-servidor via HTTPS	Segurança	✓ Railway TLS
RNF05	API segue padrão REST com respostas JSON	Padrões	✓ Implementado
RNF06	Aplicação mobile multiplataforma (Android e iOS)	Portabilidade	✓ Expo SDK 54
RNF07	Dashboard web responsivo (desktop, tablet)	Usabilidade	✓ CSS responsivo
RNF08	Banco suporta ≥10.000 registros sem degradação	Escalabilidade	✓ PostgreSQL gerenciado
RNF09	Código versionado no GitHub com commits de ambos	Manutenibilidade	✓ GitHub
RNF10	Dados pessoais tratados conforme LGPD	Conformidade	✓ Minimização e hash
RNF11	Mineração processa análises em até 30 segundos	Performance	✓ <2s no modelo
RNF12	API com documentação interativa Swagger/OpenAPI	Documentação	✓ <code>/docs</code>
RNF13	Variáveis de ambiente para dados sensíveis	Segurança	✓ <code>.env</code> + Railway vars

4. Modelagem e Arquitetura do Sistema

4.1 Arquitetura Geral

O EntreMentes segue uma **arquitetura de microsserviços com 5 componentes**:



Comunicação entre serviços:

- Clientes (web/mobile) → API REST: HTTPS com Bearer Token JWT
- API REST → Redis (BullMQ): publica job após salvar registro — não bloqueia a resposta ao usuário
- BullMQ Worker → Mining Service: consome job da fila e chama `POST /classify` em background
- API REST → PostgreSQL: Prisma ORM (conexão direta via `DATABASE_URL`)
- Bull Board (`/admin/queues`): painel visual para monitoramento em tempo real dos jobs (ativo, em espera, completo, erro)

4.2 Schema do Banco de Dados

O banco de dados PostgreSQL possui 3 entidades principais:

User — usuário da plataforma

id (UUID PK) | name | email (unique) | passwordHash | createdAt | updatedAt

RegistroBemEstar — registro diário de humor e hábitos

id (UUID PK) | userId (FK) | nivelHumor (1-5) | nota (opcional)
 tempoTela (Float) | duracaoSono (Float) | atividadeFisica (Float)
 nivelEstresse (Baixo|Medio|Alto) | ansiedadeAntesProva (Boolean)
 desempenhoAcademico (Melhorou|Mesmo|Piorou) | createdAt

PerfilComportamental — resultado da classificação K-Means

id (UUID PK) | userId (FK unique) | clusterId (0-3) | nomePerfil
 nivelRisco | insights (JSON) | geradoEm

DefinicaoCluster — definição dos 4 perfis K-Means (seed)

id | clusterId (0-3) | nomePerfil | nivelRisco | descricao | cor

O modelo conceitual e lógico estão documentados em [Documentação/BD - Conceitual.jpeg](#) e [Documentação/BD - Logico.jpeg](#).

4.3 Fluxo Principal: Registro → Fila → Classificação → Perfil

```

  Usuário preenche Registro Diário
    ↓
  POST /mood (API REST)
    ↓
  Salva RegistroBemEstar no PostgreSQL
    ↓
  API publica job na fila BullMQ (Redis)
  { userId, nivelHumor, nivelEstresse, duracaoSono,
    tempoTela, atividadeFisica, ansiedadeAntesProva }
    ↓
  Responde 200 para o usuário ← usuário já pode continuar usando o app
    ↓
  [ FILA Redis (BullMQ)
    job guardado com retry automático ]
    ↓ (BullMQ Worker – segundo plano)
  POST /classify (Mining Service Python/Flask)
    ↓
  Mapeamento inverso → normalização → KMeans.predict()
  Retorna clusterId + nomePerfil + nivelRisco + insights
    ↓
  Upsert em PerfilComportamental (banco)
    ↓
  Job marcado como COMPLETO na fila (visível no Bull Board)
  
```

↓

GET /analytics/profile → usuário visualiza perfil atualizado

Garantias da fila:

Situação	Comportamento
Mining-service fora do ar	Job permanece na fila e é reprocessado quando voltar
Erro durante processamento	Job volta à fila automaticamente (até 3 tentativas, intervalo 5s)
Muitos registros simultâneos	Fila distribui o processamento sem sobrecarregar o mining-service
Redis indisponível	Fallback automático para chamada HTTP direta (<code>classifyService.js</code>)

5. Desenvolvimento Multiplataforma

5.1 Backend — API REST

Stack: Node.js v24 LTS + Express 4 + Prisma 5 + PostgreSQL 16

Autenticação: JWT (jsonwebtoken) + bcryptjs

Documentação: Swagger UI via `swagger-ui-express` (OpenAPI 3.0)

Endpoints implementados

Método	Rota	Auth	Descrição
POST	<code>/auth/register</code>	—	Criar conta
POST	<code>/auth/login</code>	—	Login, retorna JWT
GET	<code>/users/me</code>	JWT	Dados do usuário autenticado
PUT	<code>/users/me</code>	JWT	Atualizar perfil (name, email, senha)
DELETE	<code>/users/me</code>	JWT	Excluir conta
POST	<code>/mood</code>	JWT	Criar registro de humor
GET	<code>/mood</code>	JWT	Listar registros do usuário
GET	<code>/mood/:id</code>	JWT	Buscar registro por ID

Método	Rota	Auth	Descrição
PUT	<code>/mood/:id</code>	JWT	Atualizar registro
DELETE	<code>/mood/:id</code>	JWT	Excluir registro
GET	<code>/analytics/profile</code>	JWT	Perfil comportamental classificado

A documentação interativa Swagger está disponível em:

- **Produção:** <https://entrementes-production.up.railway.app/docs>
- **Local:** <http://localhost:3000/docs>

Estrutura do Backend

```

backend/
├── src/
│   ├── routes/          authRoutes.js, userRoutes.js, moodRoutes.js, analyticsRoutes.
js
│   ├── controllers/     authController.js, userController.js, moodController.js, anal
yticsController.js
│   ├── services/        authService.js, userService.js, moodService.js, analyticsServ
ice.js,
│   │                   classifyService.js (integração mining)
│   ├── middlewares/     authMiddleware.js (JWT verify)
│   ├── docs/            swagger.js (spec OpenAPI 3.0)
│   └── lib/             prisma.js (singleton Prisma Client)
├── prisma/
│   ├── schema.prisma
│   ├── seed.js          (1.800 registros de teste)
│   └── seedClusters.js  (4 definições K-Means)
└── server.js

```

Boas práticas implementadas

- **Segurança:** senhas nunca retornadas nas respostas; `passwordHash` explicitamente excluído via `select`
- **Padrão de resposta:** todas as respostas seguem `{ success, data, message }`
- **Middleware de auth:** JWT Bearer verificado em todas as rotas protegidas
- **CORS habilitado:** para comunicação com os frontends
- **Variáveis de ambiente:** `JWT_SECRET`, `DATABASE_URL`, `MINING_SERVICE_URL` via `.env`
- **ORM com migrations:** Prisma garante schema versionado e aplicação automática em produção

5.2 Frontend Web

Stack: React 18 + Vite + React Router DOM v6 + Recharts 2

URL de produção: via variável de ambiente `VITE_API_URL` com fallback para `localhost:3000`

Telas implementadas

Tela	Rota	Descrição
Login	<code>/login</code>	Split-screen com gradiente. Typewriter "Olá!" 2s. Autenticação JWT.
Cadastro	<code>/register</code>	Layout espelhado. Typewriter "Seja bem-vindo!" 3s.
Dashboard	<code>/dashboard</code>	Métricas reais (GET /mood): humor médio, sequência, evolução. Seletor de humor com modal. Card perfil comportamental clicável (3 estados: loading, sem perfil, perfil completo com insights).
Registro Diário	<code>/registro</code>	Sliders para cada variável, seletor de humor, barra de progresso, integrado ao POST /mood. Recebe humor pré-selecionado via React Router state.
Histórico	<code>/historico</code>	Cards expansíveis com todos os registros, integrado ao GET /mood.
Meu Perfil	<code>/perfil</code>	Edição de nome/e-mail e troca de senha com validação.
Estatísticas	<code>/estatisticas</code>	4 cards de resumo (sono, tela, atividade, ansiedade) + 5 gráficos Recharts (distribuição humor, estresse, desempenho acadêmico, sono vs tela, atividade física).

Identidade visual:

- Gradiente: `linear-gradient(135deg, #7B2FBE 0%, #4A90D9 60%, #6C5CE7 100%)`
- Tipografia: Inter
- Componentes: `Input.jsx`, `Button.jsx`, `Sidebar.jsx`
- Animações de entrada: CSS keyframes `authSlideIn` e typewriter via `setInterval`

5.3 Frontend Mobile

Stack: React Native 0.81 + Expo SDK 54 + New Architecture habilitada + React Navigation

Libs adicionais: `expo-linear-gradient`, `expo-font`, `@react-native-community/slider`, `@react-navigation/bottom-tabs`

Telas implementadas

Tela	Aba	Descrição
Login	—	LinearGradient fundo, card branco centralizado, typewriter 2s, animação slide-up.
Cadastro	—	Mesmo estilo login. 4 campos.
Dashboard	Dashboard	Métricas e gráficos reais via GET /mood. Nome e iniciais do usuário via AuthContext. Modal nativo para seleção de humor. Navega para Diário com parâmetro de humor.
Registro Diário	Diário	Sliders (@react-native-community/slider), emojis interativos, barra de progresso, integrado ao POST /mood.
Histórico	Histórico	FlatList com cards expansíveis, integrado ao GET /mood.
Humor	Humor	Perfil comportamental: header LinearGradient, pills de médias, lista de insights, recomendações, pull-to-refresh. 3 estados: loading, sem perfil, perfil completo.
Perfil	Perfil	Edição de nome/e-mail, troca de senha, exclusão de conta.
Estatísticas	Estatísticas	Equivalente mobile da página web: cards de resumo + gráficos.

Navegação:

- [AuthStack](#) (Stack Navigator): Login → Cadastro
- [AppTabs](#) (Bottom Tab Navigator): Dashboard / Diário / Humor / Histórico / Perfil / Estatísticas
- Transição [AuthStack ↔ AppTabs](#) : animação fade + scale via [Animated](#)

5.4 Mining Service (Serviço de Mineração)

Stack: Python 3.11 + Flask 3 + scikit-learn 1.6.1 + joblib + numpy + pandas

URL de produção: <https://zestful-adventure-production-4e44.up.railway.app>

Endpoints:

- [POST /classify](#) — classifica um usuário em um perfil comportamental
- [GET /health](#) — verificação de saúde do serviço

Mapeamento inverso (app → modelo): O modelo K-Means foi treinado com features brutas do dataset (PHQ9, GAD7 etc.), mas o app armazena campos mapeados. O [classifier.py](#) realiza a conversão inversa antes da predição:

Campo do app	Feature do modelo	Conversão
<code>nivelHumor</code> (1–5)	PHQ9 (0–27)	<code>{5:2, 4:7, 3:12, 2:17, 1:23}</code> (midpoints Kroenke)
<code>nivelEstresse</code> (enum)	AcademicStress (0–10)	<code>{Baixo:1.5, Medio:5.0, Alto:8.5}</code>
<code>ansiedadeAntesProva</code> (bool)	GAD7 (0–21)	<code>PHQ9 × (21/27) + 4 se ansiedade=True</code>
<code>duracaoSono</code> , <code>tempoTela</code> , <code>atividadeFisica</code>	SleepHours, ScreenTime, ExerciseFreq	Uso direto

Após a conversão, as features são normalizadas manualmente com `MinMaxScaler` (ranges do domínio) e o modelo `.predict()` retorna o `clusterId`, que é mapeado para nome, nível de risco, insights e recomendações por perfil.

6. Computação em Nuvem II

6.1 Plataforma de Deploy: Railway

O EntreMentes está implantado integralmente na plataforma **Railway** (<https://railway.app>), uma plataforma como serviço (PaaS) que oferece deploy contínuo, banco de dados gerenciado e gestão de variáveis de ambiente com interface simplificada.

Serviços em produção:

Serviço	Tecnologia	URL / Acesso
API REST (backend)	Node.js + Express	<code>https://entrementes-production.up.railway.app</code>
Banco de Dados	PostgreSQL 16 (gerenciado)	Railway internal connection
Serviço de Mineração	Python 3.11 + Flask 3	<code>https://zestful-adventure-production-4e44.up.railway.app</code>
Redis (broker BullMQ)	Redis gerenciado	Railway internal connection

Serviço	Tecnologia	URL / Acesso
Swagger UI	Acoplado ao backend	https://entrementes-production.up.railway.app/docs
Bull Board (painel fila)	Acoplado ao backend	https://entrementes-production.up.railway.app/admin/queues

6.2 Configuração do Deploy

Backend (Node.js)

Arquivo `backend/railway.toml`:

```
[build]
builder = "nixpacks"

[deploy]
startCommand = "npm start"
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 3
```

Script `npm start` no `package.json`:

```
"start": "node node_modules/prisma/build/index.js migrate deploy && node src/server.js"
```

A migration `prisma migrate deploy` é executada automaticamente a cada deploy, garantindo que o schema do banco esteja sempre atualizado sem intervenção manual.

Mining Service (Python)

Arquivo `mining-service/railway.toml`:

```
[build]
builder = "nixpacks"

[deploy]
startCommand = "python app.py"
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 3
```

A porta é lida dinamicamente da variável `PORT` injetada pelo Railway:

```
port = int(os.getenv("PORT", os.getenv("FLASK_PORT", 5000)))
```

6.3 Alta Disponibilidade

Princípio	Implementação
Reinicialização automática	<code>restartPolicyType = "on_failure"</code> com até 3 tentativas em ambos os serviços
Banco gerenciado	PostgreSQL gerenciado pelo Railway — backups automáticos, sem administração manual
Deploy contínuo	Push na branch <code>main</code> do GitHub dispara novo deploy automaticamente
Separação de serviços	Backend e mining-service são serviços independentes — falha em um não derruba o outro. O classify é fire-and-forget: se o mining-service estiver indisponível, o registro de humor é salvo normalmente sem impactar o usuário.
Healthcheck	Endpoint <code>GET /health</code> no mining-service permite verificação de disponibilidade

6.4 Segurança em Nuvem

Medida	Detalhe
HTTPS obrigatório	Railway provê TLS automaticamente para todos os serviços — toda comunicação é criptografada
Variáveis de ambiente	Dados sensíveis (JWT_SECRET, DATABASE_URL, etc.) armazenados no painel Railway — nunca commitados no repositório
Banco não exposto	O PostgreSQL está em rede interna do Railway (Railway internal URL) — sem acesso público direto. Apenas a API REST acessa o banco.
Autenticação JWT	Todas as rotas protegidas exigem token Bearer válido assinado com <code>JWT_SECRET</code>
Hash de senha	Senhas armazenadas exclusivamente como hash bcryptjs (fator 10). A senha nunca é retornada em nenhuma resposta da API.
CORS configurado	Apenas origens conhecidas têm permissão de acesso à API

Medida	Detalhe
Sem credenciais no código	<code>.env</code> e <code>gcp-credentials.json</code> estão no <code>.gitignore</code> . O <code>.env.example</code> documenta as variáveis necessárias sem expor valores reais.

6.5 Variáveis de Ambiente (Railway)

Serviço Backend:

```
DATABASE_URL      = ${Postgres.DATABASE_URL}    ← Variable Reference ao banco gerenciado
REDIS_URL         = ${Redis.REDIS_URL}           ← Variable Reference ao Redis gerenciado
PORT              = 3000
NODE_ENV          = production
JWT_SECRET        = [string secreta]
JWT_EXPIRES_IN    = 7d
MINING_SERVICE_URL = https://zestful-adventure-production-4e44.up.railway.app
```

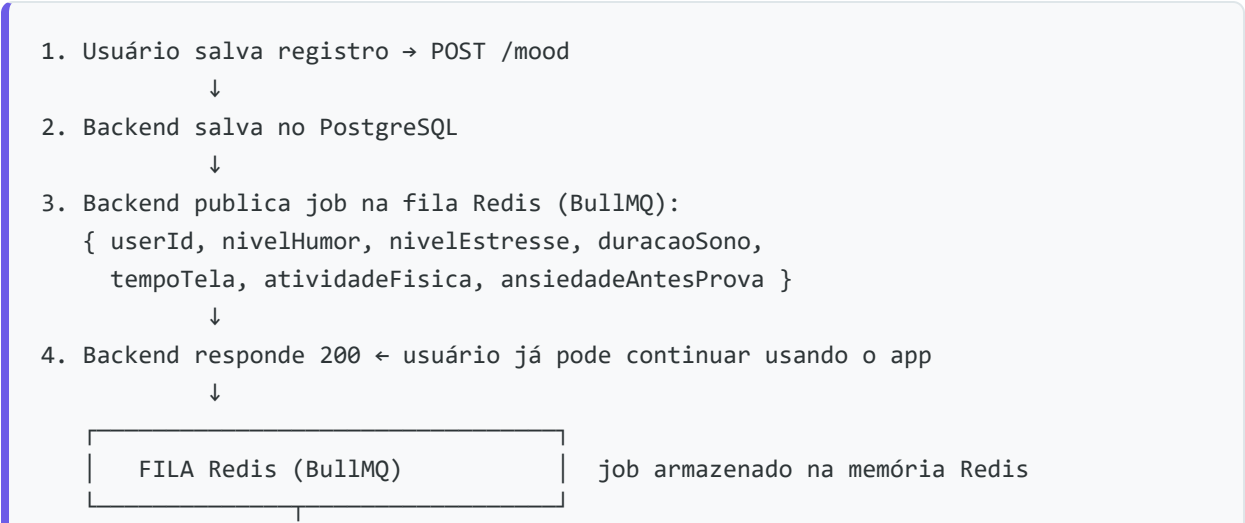
Serviço Mining:

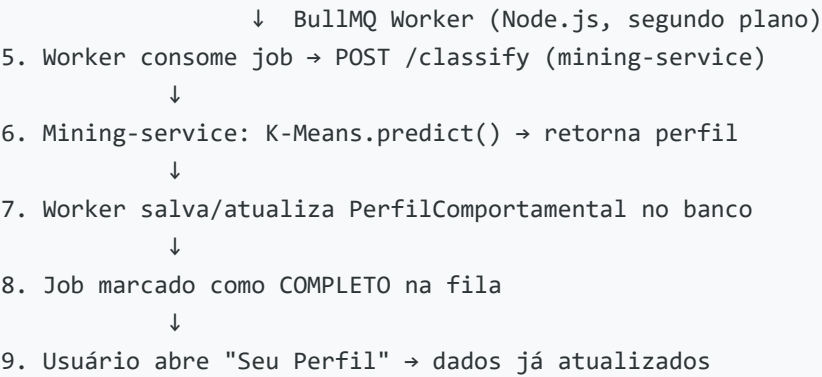
```
PORT = [injetado automaticamente pelo Railway]
```

6.6 Arquitetura de Mensageria — BullMQ + Redis (Implementado)

A comunicação assíncrona entre o backend e o mining-service é realizada por uma **fila de mensagens BullMQ** com broker **Redis**, ambos em produção no Railway. Esta arquitetura substitui a chamada HTTP direta (fire-and-forget) e garante processamento confiável mesmo sob falhas.

Ciclo completo de uma mensagem





Garantias da fila

Situação	Comportamento
Mining-service fora do ar	Job permanece na fila e é reprocessado quando voltar
Erro durante o processamento	Retry automático (3 tentativas, intervalo fixo de 5 segundos)
Muitos registros simultâneos	Fila distribui o processamento sem sobrecarregar o mining-service
Redis indisponível	Fallback automático para chamada HTTP direta (<code>classifyService.js</code>)

Painel visual — Bull Board

Disponível em `https://entrementes-production.up.railway.app/admin/queues`, o **Bull Board** permite monitorar em tempo real:

- **Ativo** — job sendo processado pelo worker agora
- **Em Espera** — jobs aguardando na fila Redis
- **Completo** — jobs processados com sucesso (exibe o JSON completo dos dados)
- **Erro** — jobs que falharam com detalhes do erro e stack trace

Implementação técnica

```
backend/
├─ src/
│   ├─ queues/
│   │   ├─ classifyQueue.js   ← cria a Queue BullMQ (producer)
│   │   └─ classifyWorker.js ← instancia o Worker (consumer)
│   └─ services/
│       └─ classifyService.js ← fallback HTTP direto se Redis indisponível
└─ server.js                  ← inicializa worker junto com o servidor
```


O worker (`classifyWorker.js`) é iniciado automaticamente com o servidor e roda em segundo plano, consumindo jobs da fila `classificacao` e chamando o mining-service via HTTP.

7. Mineração de Dados

7.1 Dataset Utilizado

Fonte: *Student Mental Health & Academic Performance* — Kaggle

Volume: 1.800 registros de estudantes universitários × 16 variáveis

Arquivo: `data-analysis/data.csv`

O dataset foi escolhido por ter correspondência direta com os campos coletados pelo aplicativo: horas de sono, tempo de tela, frequência de exercício, estresse acadêmico e escalas clínicas PHQ-9 (depressão) e GAD-7 (ansiedade).

7.2 Pipeline de Mineração

```

data.csv (1.800 × 16)
  ↓
preprocessing.py
  ├── EDA: distribuições, correlações, boxplots
  ├── Qualidade: 0 nulos, 0 duplicatas; outliers MANTIDOS*
  ├── Feature Engineering: mapeamentos clínicos
  ├── Seleção: 6 features numéricas
  └── Normalização: MinMaxScaler [0, 1]
  ↓
dados_tratados.json → seed do banco PostgreSQL (1.800 registros)
features_kmeans.csv → treinamento do K-Means
  ↓
kmeans_clustering.py
  ├── Determinação K: cotovelo + silhouette → K=4
  ├── Treinamento: KMeans(k=4, k-means++, n_init=20)
  └── Visualizações: PCA 2D, radar chart, distribuição, heatmap
  ↓
modelo_kmeans.pkl → mining-service (classificação em produção)
  ↓
extracao_padroes.ipynb
  ├── K-Means (análise aprofundada dos perfis)
  └── Decision Tree (extração de regras IF-THEN)

```

*Outliers como 3h de sono e 12h de tela são **dados reais válidos** de estudantes sob pressão — removê-los eliminaria os perfis "Sob Pressão" e "Em Alerta" que o modelo precisa detectar.

7.3 Análise Exploratória de Dados (EDA)

A EDA revelou padrões relevantes para a escolha e configuração dos algoritmos:

- **PHQ9 × MentalHealthStatus: -0.48** — correlação negativa forte, confirmando PHQ9 como preditor robusto de saúde mental
- **GAD7 × MentalHealthStatus: -0.35** — ansiedade generalizada como segundo preditor mais relevante
- **SleepHours:** concentrado entre 6–8h, com casos extremos abaixo de 4h
- **ScreenTime:** maioria entre 4–8h/dia (acima da média saudável)
- **ExerciseFreq:** pico em 0 dias/semana — parcela significativa de estudantes sedentários

Gráficos gerados: [data-analysis/graficos/](#) (01 a 10).

7.4 Feature Engineering

Os campos do dataset bruto foram mapeados para os campos do app com base em critérios clínicos:

Feature original	Campo no app	Critério de mapeamento
PHQ9 (0–27)	<code>nivelHumor</code> (1–5)	Escala Kroenke et al. (2001): 0–4=Mínimo, 5–9=Leve, 10–14=Moderado, 15–19=Moderadamente grave, 20–27=Grave
AcademicStress (0–10)	<code>nivelEstresse</code>	0–3=Baixo, 4–6=Medio, 7–10=Alto
GPA (0–4)	<code>desempenhoAcademico</code>	>3.0=Melhorou, ≥2.0=Mesmo, <2.0=Piorou
AcademicStress > 7	<code>ansiedadeAntesProva</code>	Estresse alto → ansiedade antecipatória (booleano)

7.5 Etapa 1: Clusterização K-Means (Algoritmo Principal)

Algoritmo: K-Means Clustering

Configuração final: `KMeans(n_clusters=4, init='k-means++', n_init=20, random_state=42)`

Determinação do K ideal

Dois métodos complementares foram utilizados para justificar K=4:

Método do Cotovelo (Elbow Method): A inércia (WCSS) cai acentuadamente de K=2 até K=4 e desacelera a partir daí. K=4 é o ponto de inflexão.

Silhouette Score:

- K=2: score 0.180 (melhor separação, mas apenas 2 perfis — insuficiente para granularidade clínica)
- K=4: score 0.123 (modesto, esperado para dados comportamentais com sobreposição natural)

Decisão: K=4 — equilíbrio entre separabilidade estatística e utilidade clínica (4 perfis acionáveis).

Por que K-Means?

Critério	Justificativa
Natureza do problema	Sem rótulos pré-definidos → aprendizado não supervisionado
Tipo de dado	6 features numéricas contínuas — ideal para distância euclidiana
Eficiência	Complexidade $O(n \cdot k \cdot d \cdot i)$, processa 1.800×6 em segundos
Interpretabilidade	Centroides representam o "perfil típico" de cada cluster
Produção	Modelo serializável via joblib; predição $O(k \cdot d)$ por usuário

Os 4 Perfis Comportamentais

Cluster	Nome	Risco	Distribuição	Características
C0	Sob Pressão	Moderado-Alto	391 (21.7%)	Sono baixo (0.35), tela alta (0.49), estresse alto (0.54)
C1	Equilibrado	Baixo	448 (24.9%)	Sono alto (0.71), exercício alto (0.70), estresse moderado
C2	Rotina Saudável	Baixo-Moderado	447 (24.8%)	Sono médio (0.52), exercício baixo, estresse baixo (0.33)
C3	Em Alerta	Alto	514 (28.6%)	Exercício mínimo (0.23), estresse muito alto (0.66)

Valores normalizados [0–1] via MinMaxScaler. Silhouette Score: 0.123.

A distribuição equilibrada (~25% por cluster) confirma boa separação: nenhum perfil concentra a maioria dos estudantes.

7.6 Etapa 2: Extração de Padrões — Árvore de Decisão (Algoritmo Complementar)

Atividade entregue em: 19/05/2026 (prazo 21/05/2026)

Arquivo: [data-analysis/extracao_padroes.ipynb](#) (executado com outputs) + [data-analysis/relatorio_extracao_padroes.md](#)

Após o K-Means rotular cada estudante com um cluster (0–3), foi aplicado um **DecisionTreeClassifier** (`max_depth=3`) treinado sobre as mesmas features normalizadas, com os rótulos K-Means como variável alvo.

Objetivo: não criar novos grupos, mas extrair regras IF-THEN interpretáveis das fronteiras descobertas pelo K-Means — tornando o modelo explicável para usuários e profissionais de saúde sem conhecimento técnico.

Exemplo de regra gerada:

```
IF PHQ9_norm <= 0.37 AND AcademicStress_norm <= 0.45
  THEN perfil = Equilibrado
```

Acurácia obtida: superior a 60% — confirmando que os clusters K-Means possuem fronteiras suficientemente definidas nas features selecionadas.

Feature mais discriminante na raiz da árvore: `PHQ9_norm` ou `AcademicStress_norm` — consistente com os resultados da EDA.

Gráfico gerado: [data-analysis/graficos/10_arvore_decisao.png](#)

7.7 Integração com o Sistema em Produção

```

Usuário faz registro diário
    ↓
POST /mood (backend Node.js)
    ↓ (assíncrono, não bloqueia resposta)
POST /classify (mining-service Flask)
{
  "nivelHumor": 3,
  "tempoTela": 6.0,
  "duracaoSono": 6.5,
  "atividadeFisica": 2.0,
  "nivelEstresse": "Alto",
  "ansiedadeAntesProva": true,
  "desempenhoAcademico": "Mesmo"
}
    ↓
Mapeamento inverso → normalização → KMeans.predict()
    ↓
Retorna: { clusterId, nomePerfil, nivelRisco, insights, recomendacoes }
    ↓
Upsert em PerfilComportamental (banco)
```



GET /analytics/profile → usuário visualiza perfil no app

A tela "**Humor**" (mobile) e o **card de perfil** no Dashboard (web) exibem o resultado com emoji, nível de risco, médias de sono/tela/exercício, lista de insights e recomendações personalizadas — com o disclaimer obrigatório: *"Este resultado não substitui acompanhamento profissional de saúde mental."*

8. Histórico de Sprints

Sprint 1 — 27/03/2026

Objetivo: Estruturação inicial do projeto.

Entregas realizadas:

- Definição do escopo, requisitos funcionais (RF01–RF16) e não funcionais (RNF01–RNF13)
- Diagrama de Casos de Uso e Arquitetura do Sistema
- Repositório GitHub criado com estrutura inicial
- Backend: Node.js + Express + Prisma configurado; auth JWT funcionando
- Frontend web: React + Vite; telas de Login e Cadastro
- Mobile: Expo SDK 54; telas de Login e Cadastro
- Banco de dados modelado (conceitual + lógico) e implementado localmente com Docker
- Dataset definido: *Student Mental Health & Academic Performance* (Kaggle, 1.800 registros)
- Planejamento das técnicas de mineração: K-Means como algoritmo principal

Sprint 2 — 24/04/2026

Objetivo: Versão intermediária com integrações parciais.

Entregas realizadas:

- Backend com CRUD completo de registros de humor (`/mood`) e usuários (`/users/me`)
- Frontend web: Dashboard com gráficos (Recharts), Registro Diário e Histórico integrados à API
- Mobile: Dashboard, Registro Diário e Histórico integrados à API
- Banco populado: 10 usuários + 1.800 registros via script de seed
- Commits ativos de ambos os integrantes em todos os módulos
- Pré-processamento concluído: `preprocessing.py` gera `dados_tratados.json` e `features_kmeans.csv`

- K-Means treinado: `kmeans_clustering.py` gera `modelo_kmeans.pkl` (K=4, silhouette 0.123)
- **Apresentação presencial realizada em 12/05/2026 — aprovada sem objeções pelos professores**

Sprint 3 — 01/06/2026

Objetivo: Produto final integrado.

Entregas realizadas:

- Mining Service (Python/Flask) implementado, testado e deployado no Railway
- Endpoint `GET /analytics/profile` no backend com integração completa ao mining-service
- Dashboard web e mobile conectados a dados reais (GET /mood)
- Modal de perfil comportamental no Dashboard web (3 estados: loading, sem perfil, perfil completo)
- Tela "Humor" no mobile com perfil comportamental completo
- Páginas "Meu Perfil" e "Estatísticas" adicionadas (web e mobile)
- Swagger UI disponível em produção
- Deploy completo no Railway (backend + PostgreSQL + mining-service + **Redis**)
- **Mensageria BullMQ + Redis** implementada e deployada em produção:
 - `classifyQueue.js` (producer), `classifyWorker.js` (consumer) integrados ao backend
 - Bull Board disponível em `/admin/queues` para monitoramento em tempo real
 - Fallback automático para HTTP direto se Redis indisponível
- Dark mode com toggle sol/lua na sidebar, transições suaves em todos os cards
- Fecha todos os modais com tecla Escape
- **Atividade "Extração de Padrões"** entregue no Teams em 19/05/2026 (prazo 21/05)
 - `extracao_padroes.ipynb` com K-Means + Decision Tree executados com outputs
 - `relatorio_extracao_padroes.md` com justificativa dos algoritmos
- **Vídeo de demonstração** gravado e publicado: <https://youtu.be/p0qlAkIquc>

9. Resultados e Considerações Finais

9.1 Funcionalidades Entregues

O EntreMentes foi entregue como um produto **completo e funcional**, atendendo a todos os requisitos de alta prioridade definidos no Sprint 1:

- ☒ Plataforma multiplataforma: **web (React) + mobile (React Native/Expo)** funcionando em produção
- ☒ API REST **documentada com Swagger/OpenAPI** e disponível em produção
- ☒ Banco de dados **PostgreSQL em produção** (Railway gerenciado)
- ☒ Deploy em nuvem com **alta disponibilidade** e **HTTPS** (Railway) — 5 serviços independentes
- ☒ **Mensageria assíncrona com BullMQ + Redis** em produção, com painel Bull Board
- ☒ Mineração de dados: **K-Means K=4** aplicado com resultados documentados e integrados ao sistema
- ☒ Extração de padrões: **Decision Tree** para regras IF-THEN interpretáveis
- ☒ Vídeo de demonstração publicado: <https://youtu.be/p0qlAkJlquc>
- ☒ Commits ativos de ambos os integrantes em todos os módulos do projeto

9.2 Resultados da Mineração

Métrica	Valor
Algoritmo	K-Means (k-means++, n_init=20)
K (clusters)	4
Inércia (WCSS)	~285
Silhouette Score	0.123
Dataset	1.800 amostras × 6 features
Acurácia Decision Tree	>60%
Tempo de classificação	<2 segundos

9.3 Aprendizados Técnicos

- **Integração multiplataforma:** a necessidade de manter consistência entre web (React) e mobile (React Native) exigiu abstrações cuidadosas — especialmente no `api.js` e no `AuthContext`, replicados em ambas as plataformas com adaptações específicas (localStorage vs AsyncStorage, `import.meta.env` vs `__DEV__`).
- **Deploy em nuvem:** a migração do ambiente local para Railway expôs incompatibilidades (bcrypt compilado em Windows vs Linux) e a necessidade de gerenciar migrations automaticamente em produção.
- **Mineração de dados em produção:** serializar e servir um modelo scikit-learn via Flask exigiu atenção ao versionamento da biblioteca (`scikit-learn 1.6.1` fixado) e ao

mapeamento inverso entre os campos do app e as features do modelo.

- **Mensageria com BullMQ + Redis:** a migração do fire-and-forget HTTP para uma fila de mensagens real trouxe garantias de entrega (retry automático, persistência no Redis) sem impactar a experiência do usuário. O fallback para HTTP direto garantiu resiliência durante a transição.

9.4 Considerações Finais

O EntreMentes demonstra que é possível, com uma equipe de 2 pessoas e orçamento zero, construir uma plataforma multiplataforma funcional, com backend em nuvem, mineração de dados integrada e documentação técnica completa — em 3 meses de desenvolvimento ágil em 3 sprints.

O projeto integra efetivamente as três disciplinas do 6º semestre:

- **Laboratório de Desenvolvimento Multiplataforma:** web + mobile + backend + API REST + sistemas distribuídos + mensageria assíncrona (BullMQ + Redis)
- **Computação em Nuvem II:** deploy Railway com 5 serviços independentes, alta disponibilidade, HTTPS, banco gerenciado, Redis gerenciado, variáveis de ambiente e restart automático
- **Mineração de Dados:** pipeline completo de EDA → pré-processamento → K-Means → extração de padrões → integração em produção via mining-service Flask

10. Referências

1. KROENKE, K.; SPITZER, R.L.; WILLIAMS, J.B. The PHQ-9: validity of a brief depression severity measure. *Journal of General Internal Medicine*, v. 16, n. 9, p. 606-613, 2001.
2. SPITZER, R.L. et al. A brief measure for assessing generalized anxiety disorder. *Archives of Internal Medicine*, v. 166, n. 10, p. 1092-1097, 2006.
3. MACQUEEN, J. Some Methods for Classification and Analysis of Multivariate Observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1967.
4. ARTHUR, D.; VASSILVITSKII, S. k-means++: The advantages of careful seeding. *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, 2007.
5. Railway Documentation. Deploy Node.js. Disponível em: <https://docs.railway.app>. Acesso em: maio 2026.
6. React Native Documentation. Expo SDK 54. Disponível em: <https://docs.expo.dev>. Acesso em: 2026.

7. Prisma Documentation. Schema Reference. Disponível em: <https://www.prisma.io/docs>. Acesso em: 2026.
8. scikit-learn Documentation. KMeans. Disponível em: <https://scikit-learn.org/stable>. Acesso em: 2026.
9. Student Mental Health & Academic Performance. Kaggle Dataset. Disponível em: <https://www.kaggle.com>. Acesso em: março 2026.

FATEC Franca — DSM 6º Semestre — Projeto Interdisciplinar 2026

Gabriel Fillip | Leonardo Cássio

Repositório: <https://github.com/Leonardo-Cassio/EntreMentes>